CruiseStream StudiosCover Page**Day 3**

MYNDNOVA SOLUTIONS ◆ CRUISESTREAM STUDIOS

Day 3 — AI Agents, Production & Capstone

Multi-Tool Agents · MLflow Evaluation · Production Deployment

4

Modules

3

Labs

7

Hours

Dr. Yogesh Ghorpade

Solutions Architect — AI & Data ◆ MyndNova Solutions

linkedin.com/in/yogesh-ghorpade

Today's Agenda

7

AI Agents

3 tools + tool calling
Multi-step reasoning

8

Orchestration

Supervisor / Pipeline
Specialist patterns

9

Evaluation

MLflow tracking
Production readiness

10

Capstone

End-to-end demo
Deliverables

MODULE 7 / 03

AI Agent Architecture

MODULE 7 / 04

Tool 1: SQL Lookup

```
def sql_lookup(movie_title=None, genre=None):
   """Look up movie info from Silver table."""
   query = ("SELECT title, release_year, genre, "
           "imdb_rating, box_office_tier, decade "
           "FROM ...silver_movies_clean")
   if movie_title:
       query += f" WHERE title LIKE '%{movie_title}%'"
   elif genre:
       query += f" WHERE genre = '{genre}'
   result = spark.sql(query).collect()
   return [row.asDict() for row in result]
```

■ **KEY LOGIC**Accepts `movie_title` OR `genre`. `spark.sql(query).collect()` executes SQL. `row.asDict()` converts to Python dicts for LLM.



MODULE 7 / 05

Tool 2: Plot Search

```
def plot_search(query, top_k=3):
   """Semantic search over movie plot summaries."""
   query_emb = get_embedding(query) # 1024-dim vector
   results = []
   for movie in movie_knowledge_base:
       sim = cosine_sim(query_emb, movie["embedding"])
       results.append({"title": movie["title"],
                       "genre": movie["genre"],
                       "plot": movie["summary"],
                       "similarity": round(sim, 3)})
   results.sort(key=lambda x: x["similarity"], reverse=True)
   return results[:top_k] # top 3 matches
```

■ **KEY LOGIC**Reuses Day 2's embedding + cosine similarity. `get_embedding(query)` converts query to 1024-dim. Returns top 3 semantically similar movies.

MODULE 7 / 06

Tool 3: Box Office Calculator

```python
def box_office_calc(operation, genre=None, movie_titles=None):
    """Calculate box office metrics for movies."""
    if genre:
        df = spark.sql(f"SELECT box_office_millions FROM "
            f"...silver_movies_clean WHERE genre='{genre}'")
    values = [r["box_office_millions"] for r in df.collect()]
    if operation == "total":
        return {"total_box_office": sum(values)}
    elif operation == "average":
        return {"avg_box_office": round(sum(values)/len(values), 2)}
    elif operation == "highest":
        return {"highest_grossing": max(values)}
```

■ **KEY LOGIC**Accepts `operation` (total/average/highest) + filter. Executes SQL for raw numbers, Python computes metric. Returns dict for LLM synthesis.

MODULE 7 / 07

Agent Engine: run_agent()

```
TOOL_DESCRIPTIONS = """
You have access to these tools:
1. sql_lookup(movie_title=None, genre=None) — movie info from Delta
2. plot_search(query, top_k=3) — semantic search over plots
3. box_office_calc(operation, genre=None) — total/avg/highest

Respond with JSON: {"tool": "name", "args": {"key": "value"}}
"""

def run_agent(user_request):
   # Step 1: LLM reads request + tools -> JSON plan
   plan = call_llm(TOOL_DESCRIPTIONS + user_request)
   # Step 2: Execute each tool in the plan
   for step in plan["tools"]:
       result = execute_tool(step["tool"], step["args"])
       step["result"] = result
   # Step 3: LLM synthesizes final answer
   return call_llm(f"Tool results: {plan}. Write answer.")
```

■ **KEY LOGIC**TOOL_DESCRIPTIONS tells LLM what tools exist. Step 1: LLM decides tools (JSON plan). Step 2: Python executes. Step 3: LLM writes human-friendly answer. This is the ReAct pattern.

LAB 7 / 08

Hands-On Lab

Lab 7: Test Scenarios

#	Test Scenario	Tools Used	Result
1	"Tell me about Top Gun"	sql_lookup	Pass
2	"Find movies about time travel"	plot_search	Pass
3	"Total box office for Action"	sql_lookup + box_office_calc	Pass
4	"Find sci-fi + avg revenue"	sql_lookup + plot_search + box_office_calc	Pass

SCENARIO 4 Agent plans 3 steps: SQL for Sci-Fi, plot search cross-verify, box office "average". Synthesizes: "Avg box office for Tom Cruise sci-fi movies is $370.3M." 3 tools chained autonomously.

MODULE 8 / 09

Agent Orchestration Patterns

Supervisor

```
Supervisor
 → SQL Agent
 → Search Agent
 → Calc Agent
```

One boss assigns tasks

Pipeline

```
Retrieve
 → Analyze
 → Report
```

Sequential chain

Specialist

```
SQL Agent
Search Agent
Calc Agent
(parallel)
```

Independent agents

Use Case: "Compare Action vs Drama box office + recommend best genre for marketing." Supervisor assigns SQL+calc to Data Agent, plot analysis to Content Agent, synthesizes marketing recommendation.

MODULE 9 / 10

Evaluation with MLflow

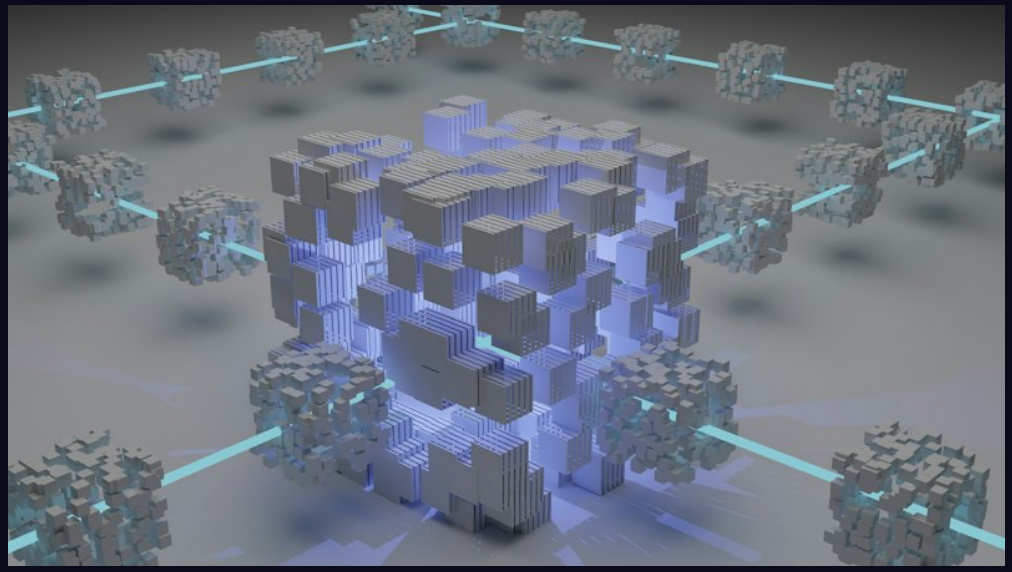
```
import mlflow

mlflow.set_experiment("cruisestream_agent_eval")

with mlflow.start_run(run_name="agent_test_4"):
   mlflow.log_param("model", "llama-3-3-70b")
   mlflow.log_param("num_tools", 3)
   mlflow.log_param("temperature", 0.3)

   for scenario in test_results:
       mlflow.log_metric(
           f"faithfulness_{scenario['id']}",
           scenario["faithfulness_score"])

   mlflow.log_metric("avg_faithfulness", 0.92)
   mlflow.log_metric("hallucination_rate", 0.0)
```



■ **KEY LOGIC** `set_experiment` creates named experiment. `start_run` begins tracking. `log_metric` records performance. Faithfulness > 0.9 = production-ready.

MODULE 9 / 11

Production Readiness Checklist

Ready (5/8)

- ✓ AI Model (Llama 3.3 70B, PAT-free)
- ✓ Agent Tools (SQL, Plot Search, Calculator)
- ✓ RAG Pipeline (retrieve+augment+generate)
- ✓ Evaluation Framework (MLflow)
- ✓ MLflow Experiment Tracking

TODO (3/8)

- ■ Model Serving Endpoint (deploy as API)
- ■ Production Monitoring (latency, errors)
- ■ Security Review (access control, audit)

Governance: 8/8 passed. Column descriptions, financial tags, masked view, RAG grounding, anti-hallucination, no raw PII exposed.

MODULE 9 / 12

Deploying to Production

```
# Step 1: Register agent in Unity Catalog
mlflow.register_model(
   model_uri="runs:/latest/cruisestream_agent",
   name="lakemeter_catalog.tom_cruise_ai.cruisestream_agent")

# Step 2: Create Serving Endpoint
from databricks.sdk import WorkspaceClient
w = WorkspaceClient()
w.serving_endpoints.create(
   name="cruisestream-agent-endpoint",
   config=EndpointConfig(
       served_models=[{"name": "cruisestream_agent",
           "model_version": "1",
           "workload_size": "Small",
           "scale_to_zero": True}]))

# Step 3: Any app calls via API
# POST /serving-endpoints/cruisestream-agent-endpoint/invocations
```

■ **KEY LOGIC**`register_model` stores agent in UC. `serving_endpoints.create` deploys as API. `scale_to_zero: True` = zero cost when idle.

MODULE 10 / 13

Capstone Project — 5 Phases

PHASE 1

Architecture Summary
S3 → Databricks → AI

PHASE 2

Data Verification
Bronze, Silver, Gold, Views

PHASE 3

Agent Demo
3 multi-tool scenarios

PHASE 4

Governance + Evaluation
8/8 checks, 5/8 readiness

PHASE 5

Enterprise Presentation
Full deliverable showcase

MODULE 10 / 14

Complete Technology Stack

Layer	Technology	Role
Storage	AWS S3	Raw movie data
Security	AWS IAM	Access control
Platform	Databricks	Delta + UC + AI
Format	Delta Lake	ACID, time travel
Architecture	Medallion	Bronze/Silver/Gold
Governance	Unity Catalog	Masking, tagging
GenAI	Llama 3.3 70B	Summarization, Q&A
Embeddings	GTE Large 1024-dim	Semantic search
RAG	Retrieve+Augment+Generate	Grounded answers
Agent	3-Tool Multi-Step	SQL, search, calc
Evaluation	MLflow	Tracking, metrics

MODULE 10 / 15

Enterprise Use Cases & Scenarios

Viewer Query (Multi-Tool)

"Find Tom Cruise movies about brothers, total box office." Agent chains: plot_search("brothers") → Rain Man → sql_lookup → box_office_calc("total") → "Rain Man grossed $354.8M."

Executive Dashboard

"Average box office per genre?" Agent: sql_lookup for each genre → box_office_calc("average") → Action $357M, Drama $354.8M, Sci-Fi $370.3M.

Content Curation

"Find movies similar to Top Gun." Agent: plot_search("fighter pilot mission") → Top Gun: Maverick (0.91) + Edge of Tomorrow (0.68) → recommends "Military Action" collection.

Compliance Audit

"Can AI access raw financial data?" Agent queries v_movies_ai_safe → sees "Blockbuster" not $591M. 8/8 governance checks pass.

RECAP / 16

3-Day Learning Journey

Day 1

Foundation

- AI vs GenAI vs Agents
- S3 + IAM architecture
- Delta Lake ACID + time travel
- Bronze/Silver/Gold SQL
- Column masking function
- AI-safe masked views

Day 2

GenAI & RAG

- mlflow.deployments (PAT-free)
- Summarize/Q&A/Classify
- 1024-dim GTE embeddings
- cosine_sim semantic search
- rag_answer() 3-step pipeline
- Anti-hallucination 3/3

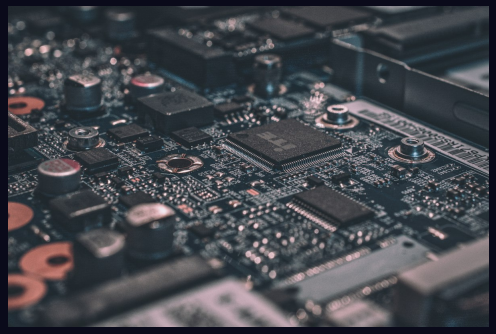
Day 3

Agents & Production

- 3 tool functions (SQL/search/calc)
- TOOL_DESCRIPTIONS + run_agent
- 4 test scenarios all pass
- MLflow evaluation logging
- Production 5/8 + governance 8/8
- Model serving endpoint

Thank You!

CruiseStream Studios — End-to-End Enterprise AI on Databricks



12

Movies

3

Tables

3

Tools

48

Quiz Qs

Dr. Yogesh Ghorpade

Senior Solutions Architect — AI & Data

MyndNova Solutions ◆ Databricks Certified

linkedin.com/in/yogesh-ghorpade